

# EE 446: TinyML

## Assignment 1: Discussion Answers ([Dis])

### Student Information

Full Name: Luke Valerio

Student ID: 1901711

## Problem 1 (e): Further Model Size Reduction [Pro/Dis]

### 1. Comparison of the models from parts (a) to (d).

All of the compression techniques kept classification quality high. Every model reached 100% test accuracy on the Wine dataset, which is small and easy to separate, so the models differ mainly in size. Among the techniques in parts (b), (c), and (d), the knowledge-distilled student was the smallest at 6.12 KB. Distillation shrinks the actual architecture (1,027 parameters versus 3,075 in the baseline) rather than just re-encoding the same weights.

| Model                | Technique (part) | Size (KB) | Test acc. |
|----------------------|------------------|-----------|-----------|
| Baseline (float32)   | baseline (a)     | 14.08     | 100%      |
| Dynamic-range quant. | quantization (b) | 8.55      | 100%      |
| Float16 quant.       | quantization (b) | 8.95      | 100%      |
| Full INT8 quant.     | quantization (b) | 8.00      | 100%      |
| Pruned               | pruning (c)      | 14.11     | 100%      |
| KD student           | distillation (d) | 6.12      | 100%      |
| KD + INT8            | combined (e)     | 4.77      | 100%      |

Table 1. Model size and test accuracy across all compression techniques. The combined KD plus INT8 model in part (e) is the smallest.

A notable result is that pruning did not reduce the file size. The pruned model is 14.11 KB, essentially the same as the 14.08 KB baseline. Magnitude pruning sets small weights to zero but does not change the shape of the network, and the standard TFLite flatbuffer stores those zeros densely, so the file is no smaller. Size savings from pruning only appear when a sparse storage format or sparsity-aware compression is used.

### 2. Proposed strategy.

Yes, the model can be made smaller than any single technique in parts (b) to (d) by combining knowledge distillation with full INT8 quantization. Distillation first reduces the number of parameters through the smaller 32-16-3 student, and INT8 quantization then stores each remaining weight in one byte instead of four. The two techniques act on different axes, fewer weights versus cheaper weights, so their savings add together.

### 3. Implementation.

The trained student model from part (d) was passed through the same full-integer quantization pipeline used in part (b), with default optimizations, a representative dataset for activation calibration, and int8

input and output. This produced the file `model_kd_int8.tflite`. See the corresponding cell in the [Pro] notebook.

#### 4. Evaluation.

The combined model is 4.77 KB and still reaches 100% test accuracy. That is roughly three times smaller than the 14.08 KB baseline, and smaller than every model from parts (b), (c), and (d). The next smallest was the 6.12 KB KD student.

#### 5. Justification and the biggest factor.

Knowledge distillation provided the larger structural reduction by cutting the parameter count by about two thirds. INT8 quantization then delivered a further reduction of about four times in per-weight storage on top of that smaller model. Applying INT8 to the distilled student was the single change that pushed the size below every other model. Accuracy did not drop because the Wine task is simple enough that the reduced-precision, reduced-capacity model still separates the three classes perfectly. On a harder dataset we would expect a small accuracy trade-off at this level of compression.

---

## Problem 2: Exploring Edge Impulse [Dis]

### 1. Learning Blocks available in the free tier (with example models).

In Edge Impulse the learning block is the model stage of an impulse. In the free (developer) account, the Add a learning block dialog lists the officially supported blocks below, which cover both supervised and unsupervised learning.

| Learning block                               | Paradigm     | What it does, and an example model                                                                                                                                      |
|----------------------------------------------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Classification                               | Supervised   | Learns patterns and sorts inputs into classes; good for categorizing movement or recognizing audio. Example: a keyword-spotting model that outputs yes, no, or unknown. |
| Regression                                   | Supervised   | Learns patterns to predict numeric, continuous values. Example: estimating a temperature or a count from sensor readings.                                               |
| Transfer Learning (Images)                   | Supervised   | Fine-tunes a pre-trained image model on your data and works well even with small image datasets. Example: a cat versus dog image classifier.                            |
| Transfer Learning (Keyword Spotting)         | Supervised   | Fine-tunes a pre-trained audio model to your own keywords. Example: a wake-word detector.                                                                               |
| Object Detection (FOMO, MobileNetV2 SSD FPN) | Supervised   | Locates and classifies objects within an image. Example: FOMO counting objects on a conveyor belt.                                                                      |
| Anomaly Detection                            | Unsupervised | Finds outliers; data that is unlikely under the model is flagged as anomalous. Example: detecting abnormal machine vibration from an accelerometer.                     |
| Visual Anomaly Detection (FOMO-AD)           | Unsupervised | Detects visual anomalies using a pre-trained backbone and a scoring function, and does not require anomalous data. Example: spotting defects on a product line.         |

The dialog also includes a Classical ML block, and, as its banner notes, you can bring your own model built in PyTorch, Keras, or scikit-learn. Self-supervised learning is not offered as a dedicated free-tier learning block.

## ***2. Layer types available in the Classifier, with their purpose.***

| Layer                   | Purpose                                                                                                                     |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Input                   | Receives the incoming feature data and performs no computation.                                                             |
| Dense (fully connected) | Every neuron connects to all neurons of the previous layer; used for general-purpose learning and the final classification. |
| 1D Convolution          | Applies filters across sequential data such as audio or time-series to extract local features.                              |
| 1D Pooling              | Downsamples 1D feature maps using max or average pooling to reduce size and add robustness.                                 |
| 2D Convolution          | Applies filters across an image's height and width to detect spatial patterns.                                              |
| 2D Pooling              | Downsamples 2D feature maps to reduce their spatial dimensions.                                                             |
| Flatten                 | Converts multi-dimensional data into a one-dimensional vector, usually before a Dense layer.                                |
| Reshape                 | Changes the shape of the data without altering its values, to fit a layer's expected input.                                 |
| Dropout                 | Randomly sets a fraction of units to zero during training as regularization to reduce overfitting.                          |
| Output                  | Produces the final class scores from the learned features.                                                                  |

Switching the block to Expert mode exposes the full Keras API, so other layer types such as Batch Normalization can also be added beyond the default visual editor set.

## ***3. Model compression options under Deployment.***

Edge Impulse offers compression along two independent axes at deployment.

### ***Model precision (Model optimizations):***

| Option                | Description                                                                                                    |
|-----------------------|----------------------------------------------------------------------------------------------------------------|
| Quantized (int8)      | Converts 32-bit float weights and activations to 8-bit integers, about 75% smaller with minimal accuracy loss. |
| Unoptimized (float32) | Full-precision model, the largest size and highest fidelity.                                                   |

### ***Compiler and runtime:***

| Option                                        | Description                                                                            |
|-----------------------------------------------|----------------------------------------------------------------------------------------|
| TensorFlow Lite / LiteRT for Microcontrollers | The standard interpreter-based runtime.                                                |
| EON Compiler                                  | Runs the same network using less RAM and flash with the same accuracy as LiteRT Micro. |
| EON Compiler (RAM-optimized)                  | Reduces RAM usage even further when possible.                                          |

Internally the EON Compiler applies int8 quantization, pruning, and operator fusion, and it reports a before and after comparison of latency, RAM, and flash.

#### ***4. Synthetic data generation: modalities, methods, and importance.***

| Modality                        | Generation method or tool (algorithm)                                  | Tier               |
|---------------------------------|------------------------------------------------------------------------|--------------------|
| Images                          | DALL-E (OpenAI, text to image); Flux Pro (Replicate, text to image)    | Free (own API key) |
| Audio, speech and keywords      | Whisper Keyword Spotting generation block (OpenAI)                     | Free (own API key) |
| Audio, sound effects and events | ElevenLabs Synthetic Audio block, for example glass breaking or alarms | Free (own API key) |
| Time-series                     | Time-series Data Augmentation block (Edge Impulse)                     | Enterprise only    |

#### ***Why synthetic data is important:***

Synthetic data fills gaps where real data is scarce, expensive, dangerous, or privacy-sensitive to collect, such as rare events, anomalies, and edge cases like the sound of glass breaking. It lets you balance classes and augment small datasets, which improves a model's ability to generalize and stay robust. It also allows a project to be started and validated before a full real-world dataset has been gathered, which saves a large amount of collection and labeling time and cost.